

Installation

🔥 cyberforge.blog/writeups/GOAD/Installation

This page is heavily inspired by the official GOAD installation page, found in this [Github](#)

Dependencies and recommendations

Dependencies:

Dependency	Version or note
Vagrant	Version 2.2.18 -> Used to create the VMs
VirtualBox	Version 6.1 not 7 -> Hypervisor
GOAD Github	All GIT GOAD files
Linux Server	Hardware recommendations are just below

Hardware dependencies:

Hardware	Value
CPU	Around 16 Cores (excluding Host OS)
RAM	32Gb of RAM (16Gb is possible but with compromises)
HDD	120Gb for the LAB itself!

Preparation and installation of dependencies

1. Let's start by cloning the repo

```
git clone https://github.com/Orange-Cyberdefense/GOAD.git
```

The downloaded folder structure should look like this:

```
drwxrwxrwx 1 victorhin0 victorhin0 0 26 juin 13:47 ad
drwxrwxrwx 1 victorhin0 victorhin0 4,0K 27 juin 11:48 ansible
-rwxrwxrwx 1 victorhin0 victorhin0 399 26 juin 13:47 Dockerfile
drwxrwxrwx 1 victorhin0 victorhin0 0 26 juin 13:47 docs
-rwxrwxrwx 1 victorhin0 victorhin0 35K 26 juin 13:47 LICENSE
drwxrwxrwx 1 victorhin0 victorhin0 90 26 juin 13:47 providers
-rwxrwxrwx 1 victorhin0 victorhin0 31K 26 juin 13:47 README.md
drwxrwxrwx 1 victorhin0 victorhin0 0 26 juin 13:47 vagrant
-rwxrwxrwx 1 victorhin0 victorhin0 4,8K 26 juin 13:47 Vagrantfile
```

Make sure the Vagrant file and the ansible and ad folders are present

Install Virtualbox

The version of VirtualBox we are going to install is not the latest! You can find all the installers on this page: https://www.virtualbox.org/wiki/Download_Old_Builds_6_1

I recommend using the "All distributions" package which will compile and install VirtualBox

▼ Important if you install GOAD in a VM

It is important to enable virtualization in the virtual machine menu

In my case, my machine is a VM in an ESX (Vmware) so I had to enable virtualization like this:

CPU		20	
Cœurs par socket	1	Sockets : 20	
Connexion CPU à chaud	☐ Activer l'ajout à chaud du CPU		
Réservation	0	MHz	
Limite	Illimité	MHz	
Parts	Normales	20000	
Masque CPUID	Exposer l'indicateur NX/XD à l'invité Avancé...		
Virtualisation matérielle	☒ Exposer l'assistance matérielle à la virtualisation au SE invité		

Install Vagrant

Vagrant will be used to create the empty VMs on VirtualBox.

To install Vagrant, we need to download the .deb file corresponding to version **2.12.18**

This .deb can be found here:

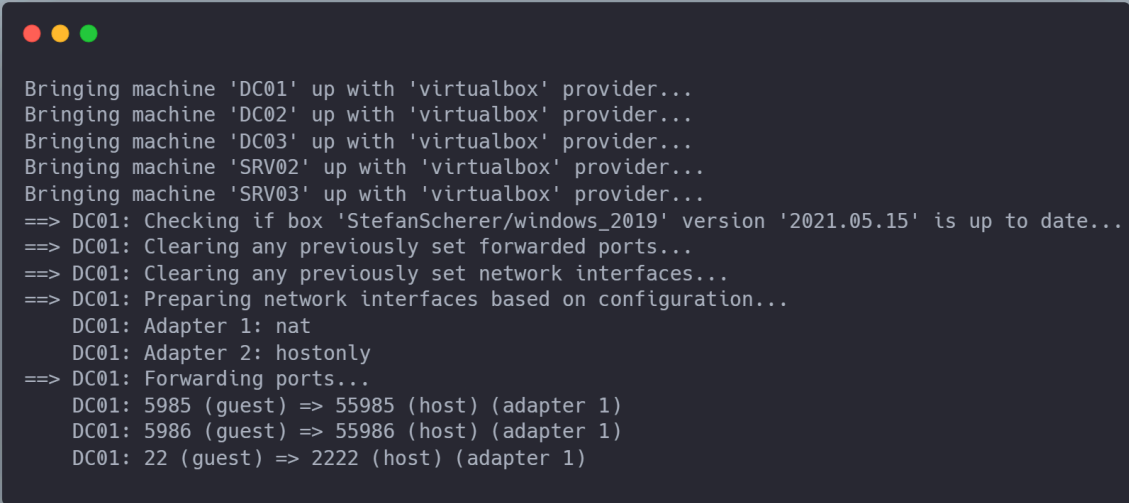
https://releases.hashicorp.com/vagrant/2.2.18/vagrant_2.2.18_x86_64.deb

Once this deb is downloaded, just install it

Creation of virtual machines

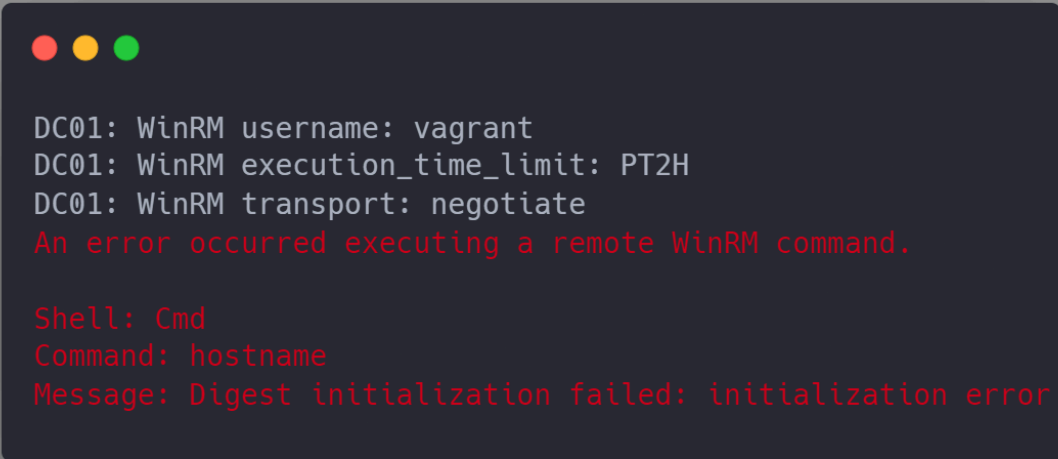
We just have to launch the VM creation with this command:

You should have an output like this:



```
Bringing machine 'DC01' up with 'virtualbox' provider...
Bringing machine 'DC02' up with 'virtualbox' provider...
Bringing machine 'DC03' up with 'virtualbox' provider...
Bringing machine 'SRV02' up with 'virtualbox' provider...
Bringing machine 'SRV03' up with 'virtualbox' provider...
==> DC01: Checking if box 'StefanScherer/windows_2019' version '2021.05.15' is up to date...
==> DC01: Clearing any previously set forwarded ports...
==> DC01: Clearing any previously set network interfaces...
==> DC01: Preparing network interfaces based on configuration...
DC01: Adapter 1: nat
DC01: Adapter 2: hostonly
==> DC01: Forwarding ports...
DC01: 5985 (guest) => 55985 (host) (adapter 1)
DC01: 5986 (guest) => 55986 (host) (adapter 1)
DC01: 22 (guest) => 2222 (host) (adapter 1)
```

If you have the following error:



```
DC01: WinRM username: vagrant
DC01: WinRM execution_time_limit: PT2H
DC01: WinRM transport: negotiate
An error occurred executing a remote WinRM command.

Shell: Cmd
Command: hostname
Message: Digest initialization failed: initialization error
```

Make sure you have vagrant version 2.2.18. You can check the version with the following command:

On the first vagrant UP you do, the different ISOs will be downloaded. From the second time, you won't have to download them anymore, they will be cached!

Once the VM creation is done, you should have the 5 machines (DC01 to 3 then SRV2 and 3) turned on and properly created on VirtualBox. We can then move on to provisioning these machines on Ansible

Provisioning machines and building AD with Ansible

Ansible installation

Before provisioning the machines with Ansible, we must install some dependencies. Here you have 2 choices:

1. The server/VM you are using is dedicated to GOAD

In which case, you can directly install the python libraries without going through a virtual environment:

```
cd GOAD/ansible
python3 -m pip install --upgrade pip
python3 -m pip install ansible-core==2.12.6
python3 -m pip install pywinrm
ansible-galaxy install -r requirements.yml
```

2. The server/VM you are using is multipurpose, not just for GOAD

In which case you will have to go through a virtual environment:

```
python3 -m virtualenv .venv
source .venv/bin/activate

python3 -m pip install --upgrade pip
python3 -m pip install ansible-core==2.12.6
python3 -m pip install pywinrm

ansible-galaxy install -r requirements.yml
```

It happens that python libraries conflict with other libraries. We use virtual environments for this.

In general, we have one virtual environment per project. We can even use environment managers, such as conda: <https://anaconda.org/anaconda/conda>

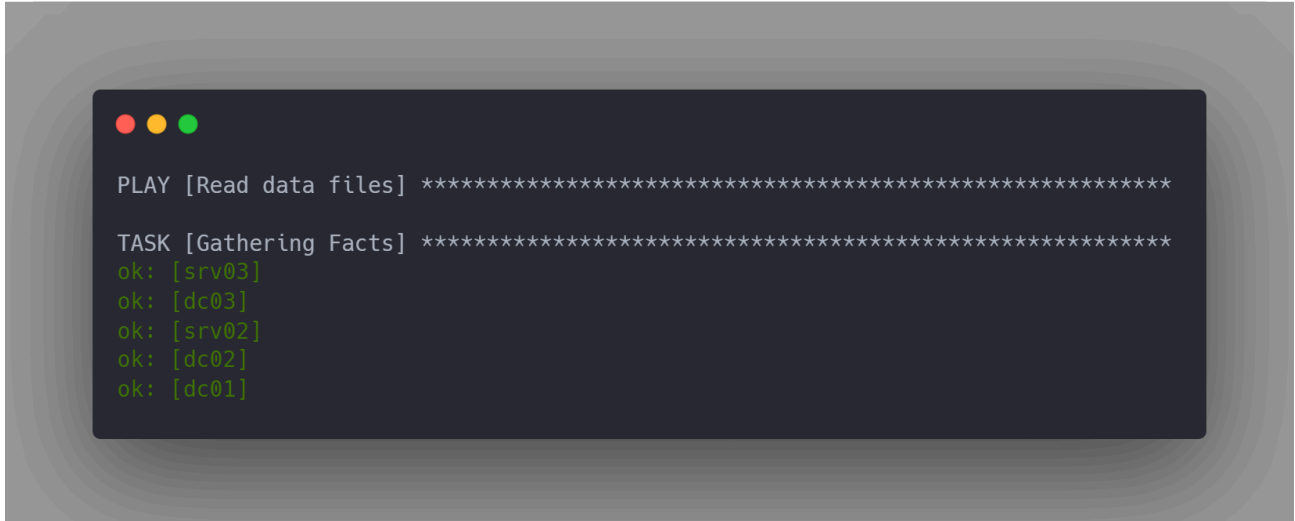
Moreover, it is generally recommended to go through a virtual environment even if there is no other project.

Provisioning machines with Ansible

Puis ensuite nous pouvons commencer à provisionner les VM via cette commande :

```
ansible-playbook -i ../ad/sevenkingdoms.local/inventory main.yml
```

When launching the script, this is what should appear

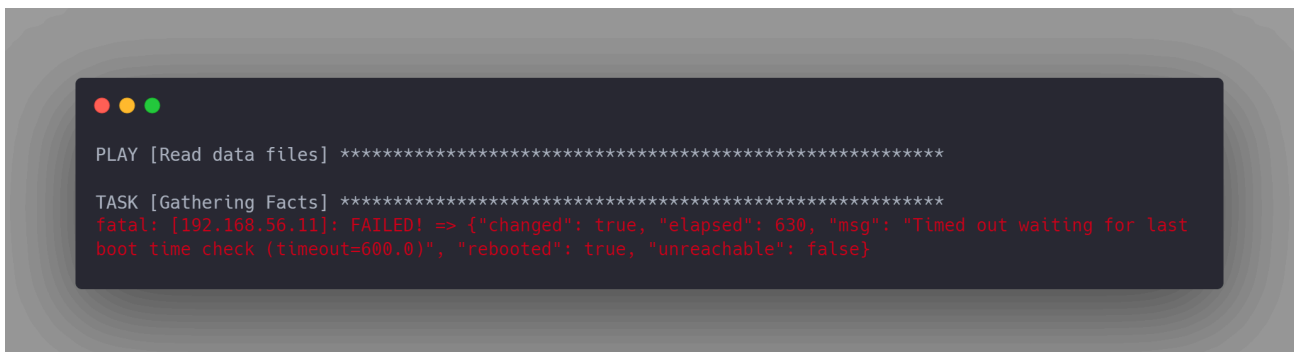
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal output shows the execution of an Ansible playbook. It starts with 'PLAY [Read data files]' followed by a line of asterisks. Then it shows 'TASK [Gathering Facts]' followed by another line of asterisks. Below this, there are five lines of output, each starting with 'ok:' followed by a list of hostnames in brackets: [srv03], [dc03], [srv02], [dc02], and [dc01].

```
PLAY [Read data files] *****

TASK [Gathering Facts] *****
ok: [srv03]
ok: [dc03]
ok: [srv02]
ok: [dc02]
ok: [dc01]
```

▼ In case there is an error message

Case #1:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The terminal output shows the execution of an Ansible playbook. It starts with 'PLAY [Read data files]' followed by a line of asterisks. Then it shows 'TASK [Gathering Facts]' followed by another line of asterisks. Below this, there is a line of output in red text indicating a failure: 'fatal: [192.168.56.11]: FAILED! => {"changed": true, "elapsed": 630, "msg": "Timed out waiting for last boot time check (timeout=600.0)", "rebooted": true, "unreachable": false}'.

```
PLAY [Read data files] *****

TASK [Gathering Facts] *****
fatal: [192.168.56.11]: FAILED! => {"changed": true, "elapsed": 630, "msg": "Timed out waiting for last boot time check (timeout=600.0)", "rebooted": true, "unreachable": false}
```

This means that the VMs are updating, and therefore, the ansible script has "abandoned" the idea of provisioning the VMs

Wait for the VMs to finish booting, then:

Case #2:

```
PLAY [Read data files] *****

TASK [Gathering Facts] *****
fatal: [192.168.56.22]: UNREACHABLE! =>
{"changed":false,"msg":"ssl:HTTPConnectionPool(host='192.168.56.22', port=5986): Max retries exceeded with url: /wsman (Caused by NewConnectionError('<urllib3.connection.HTTPSConnection object at 0x10ad68ee0>: Failed to establish a new connection: [Errno 61] Connection refused'))","unreachable":true}
```

This error often happens, and usually affects 1 or 2 VMs. You must re-provision the affected VMs with vagrant.

In order not to waste too much time, it is possible to provision the VMs individually with the following command:

Finally, a report will be presented to you.

```
PLAY RECAP *****
dc01                : ok=99   changed=55   unreachable=0   failed=0   skipped=18   rescued=0
ignored=0
dc02                : ok=43   changed=24   unreachable=0   failed=1   skipped=8    rescued=0
ignored=0
dc03                : ok=107  changed=66   unreachable=0   failed=0   skipped=11   rescued=0
ignored=0
srv02               : ok=108  changed=62   unreachable=0   failed=0   skipped=9    rescued=0
ignored=0
srv03               : ok=97   changed=55   unreachable=0   failed=0   skipped=8    rescued=0
ignored=0
```

For example, here we see that there was an error on DC02. So I would have to run the ansible script again to fix this error!

Snapshotting virtual machines with Vagrant

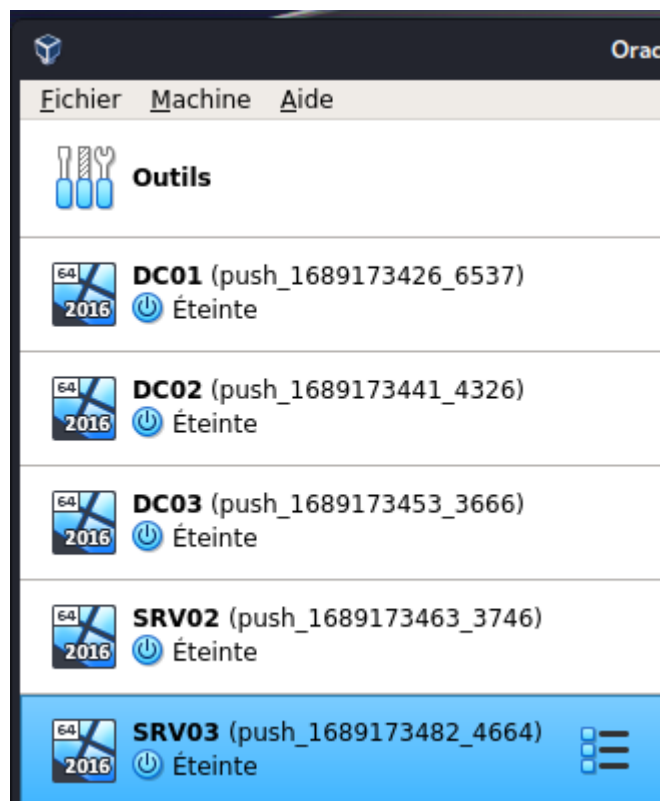
Once this is done, it is possible to take a snapshot of the VMs with vagrant:

```

==> DC01: Snapshotting the machine as 'push_1689173426_6537'...
==> DC01: Snapshot saved! You can restore the snapshot at any time by
==> DC01: using `vagrant snapshot restore`. You can delete it using
==> DC01: `vagrant snapshot delete`.
==> DC02: Snapshotting the machine as 'push_1689173441_4326'...
==> DC02: Snapshot saved! You can restore the snapshot at any time by
==> DC02: using `vagrant snapshot restore`. You can delete it using
==> DC02: `vagrant snapshot delete`.
==> DC03: Snapshotting the machine as 'push_1689173453_3666'...
==> DC03: Snapshot saved! You can restore the snapshot at any time by
==> DC03: using `vagrant snapshot restore`. You can delete it using
==> DC03: `vagrant snapshot delete`.
==> SRV02: Snapshotting the machine as 'push_1689173463_3746'...
==> SRV02: Snapshot saved! You can restore the snapshot at any time by
==> SRV02: using `vagrant snapshot restore`. You can delete it using
==> SRV02: `vagrant snapshot delete`.
==> SRV03: Snapshotting the machine as 'push_1689173482_4664'...
==> SRV03: Snapshot saved! You can restore the snapshot at any time by
==> SRV03: using `vagrant snapshot restore`. You can delete it using
==> SRV03: `vagrant snapshot delete`.

```

This will create snapshots directly integrated into VirtualBox:



And there you go, your environment is ready! We can start attacking it!